L'héritage

Tous droits réservés à STUDI - Reproduction interdite

Table des matières

I. Contexte **3**

II. L'héritage **3**

A. L'héritage 3

B. Exercice : Quiz 5

III. Le polymorphisme **6**

A. Le polymorphisme 6

B. Exercice : Quiz 8

IV. Notions avancées d'héritage **10**

A. Les types d'héritage 10

V. L'essentiel **11**

VI. Auto-évaluation **11**

A. Exercice 11

B. Test 12

Solutions des exercices **13**

Tous droits réservés à STUDI - Reproduction interdite

I. Contexte

Durée (en minutes) : 60

Environnement de travail : Python 3.12+, PyCharm Community

Pré requis : bases en Python

Contexte

Un aspect qui distingue Python est son approche cohérente et structurée de la programmation orientée objet.

Un programme Python adopte une architecture précise qui favorise la modularité et l'extensibilité.

Maîtriser cette structure est fondamental pour mettre en œuvre vos projets.

Dans ce cours, nous allons découvrir plusieurs concepts fondamentaux à la programmation orientée objet.

Plus précisément, nous allons nous concentrer sur l'héritage et le polymorphisme en Python, deux concepts clés étroitement liés pour la conception de logiciels modulaires et réutilisables. La compréhension et l'application de ces concepts sont des compétences essentielles à développer pour travailler efficacement avec Python.

II. L'héritage

A. L'héritage

Définition **L'héritage**

En programmation orientée objet, l'héritage est un mécanisme qui permet à une classe d'acquérir les propriétés (attributs et méthodes) d'une autre classe dès son instanciation. La classe qui est héritée est souvent appelée classe parent, classe mère ou encore superclasse, tandis que la classe qui hérite est appelée classe enfant ou sous-classe. L'héritage favorise la réutilisation du code et la modularité, car les classes enfants peuvent réutiliser et adapter le code des classes parents selon leurs besoins.

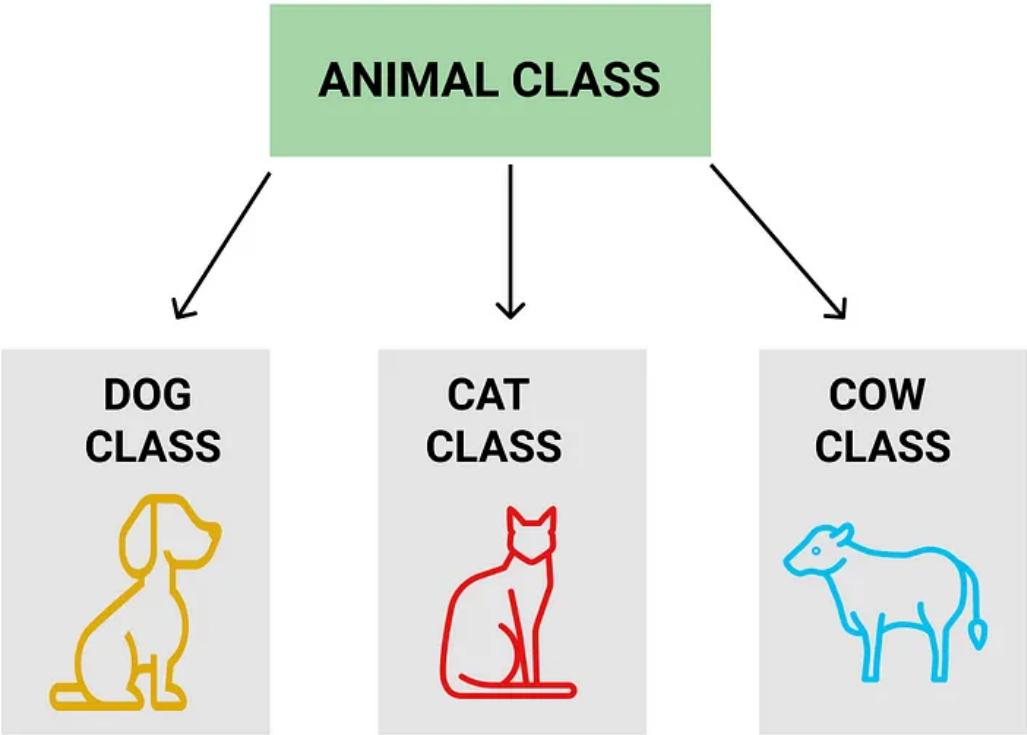
L'utilité de l'héritage

L'héritage en Python, comme dans d'autres langages de programmation orientés objet, offre deux avantages principaux : la réutilisation du code et l'organisation du code.

La réutilisation du code est possible grâce aux classes enfants qui héritent des attributs et des méthodes des classes parents, ce qui signifie que vous n'avez pas besoin de réécrire ce code dans la classe enfant.

De même, l'héritage permet de créer une hiérarchie de classes qui reflète la relation « *est un type de* » entre différents objets, ce qui permet d'avoir un code mieux organisé.

L'image suivante illustre le concept : les classes « *dog* », « *cat* » et « *cow* » héritent toutes les trois d'une seule classe mère « *animal* ».



Source : Medium[1]

Évidemment, les méthodes et attributs ne seront pas tous identiques. Vous n'imaginez pas une vache produire le même son qu'un chien !

C'est pourquoi les méthodes de chaque classe pourront être redéfinies, ou autrement dit « *personnalisée* ». La surcharge est une notion fondamentale que nous avons eu l'occasion de voir lors du cours sur les méthodes spéciales, notamment.

Rappel **La surcharge**

La surcharge est une technique courante en Python. Surcharger une méthode, c'est simplement en personnaliser son comportement, qui sera propre à notre classe. Par hiérarchisation, la définition personnalisée de notre fonction sera toujours prioritaire par rapport à l'implémentation « *de base* » de la fonction dans la classe mère.

Exemple **L'héritage et la surcharge**

```
class Animal:
    nbrOfLegs = 0
    canSeeInTheDark = False

    def __init__(self):
        self.nbrOfLegs = 4
        self.canSeeInTheDark = True

    def speak(self):
        print("not defined!")

```

1 https://akshayraut.medium.com/inheritance-in-object-oriented-programming-8c61b93ca5a8

Tous droits réservés à STUDI - Reproduction interdite

```python
class Dog(Animal):

    def __init__(self):
        super().__init__()

    def speak(self):
        print("Wouaf wouaf!")

class Cat(Animal):
    def __init__(self):
        super().__init__()

    def speak(self):
        print("Miaouw!")


class Cow(Animal):
    def __init__(self):
        super().__init__()
        self.canSeeInTheDark = False

    def speak(self):
        print("Meuuuuhhh!")

genericAnimal = Animal()

print(genericAnimal.nbrOfLegs)
print(genericAnimal.canSeeInTheDark)

Laika = Dog()

Laika.speak()
print(Laika.canSeeInTheDark)

Kitty = Cat()

Kitty.speak()
print(Kitty.canSeeInTheDark)

Marguerite = Cow()

Marguerite.speak()
print(Marguerite.canSeeInTheDark)
```

B. Exercice : Quiz

[solution n°1 p.15]

Question 1

Qu'est-ce que l'héritage en programmation orientée objet ?

- ○ Un mécanisme permettant de créer des classes
- ○ Un mécanisme permettant à une classe d'acquérir les propriétés et méthodes d'une autre classe
- ○ Un mécanisme permettant d'ajouter des attributs à une classe
- ○ Un mécanisme permettant d'ajouter des méthodes à une classe

Tous droits réservés à STUDI - Reproduction interdite

Question 2

Quels sont les avantages principaux de l'héritage en Python ?

- ☐ Une meilleure performance d'exécution
- ☐ La modularité
- ☐ La réutilisation du code et sa structuration
- ☐ La simplification du code

Question 3

Comment une classe hérite-t-elle d'une autre classe en Python ?

- ○ En utilisant le mot-clé 'extends'
- ○ En indiquant la classe parent entre parenthèses dans la définition de la nouvelle classe
- ○ En utilisant le mot-clé 'inherit'
- ○ En indiquant la classe parent après un double point dans la définition de la nouvelle classe

Question 4

Qu'est-ce que la surcharge de méthode en Python ?

- ○ Modifier le comportement d'une méthode héritée dans la classe enfant
- ○ Ajouter une nouvelle méthode à une classe existante
- ○ Supprimer une méthode d'une classe existante
- ○ Rendre une méthode héritée privée dans la classe enfant

Question 5

Comment fait-on référence aux méthodes ou aux attributs du parent dans la classe enfant ?

- ○ Ce n'est pas possible
- ○ En indiquant le nom de la classe parent suivi d'un point : 'class.__'
- ○ Cela n'est pas nécessaire, on peut simplement écrire le nom de la méthode ou de l'attribut
- ○ En utilisant le mot clé 'super()'

III. Le polymorphisme

A. Le polymorphisme

Définition **Le polymorphisme**

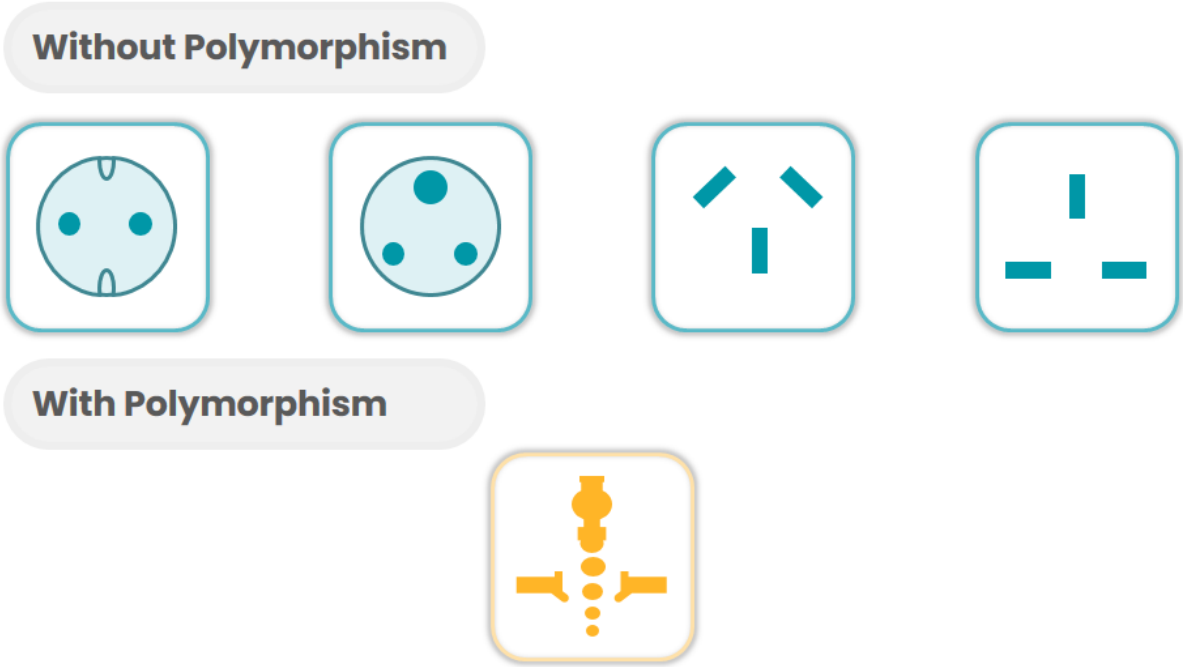
Le polymorphisme est un des concepts fondamentaux de la programmation orientée objet. Il décrit la capacité d'un objet à prendre littéralement plusieurs formes. Par ailleurs, selon ce concept, différents appels à une même méthode peuvent générer différents types de résultats, en fonction du type d'arguments fournis lors de l'appel. On distingue ainsi le polymorphisme d'objets et le polymorphisme de fonctions.

Tous droits réservés à STUDI - Reproduction interdite

Le polymorphisme d'objets

En réalité, nous avons déjà appliqué la notion de polymorphisme dans l'exemple précédent. Le chien, le chat et la vache sont *des formes* d'animaux, c'est un parfait exemple de polymorphisme.

Un second exemple qui illustre bien ce concept de polymorphisme est le suivant :



Ainsi, si l'on voulait définir un objet « *prise de courant* », sans polymorphisme il faudrait définir autant de classes d'objets distincts qu'il existe de prises de courants. Avec le polymorphisme, on peut définir une prise de courant « *universelle* », capable de représenter l'ensemble des prises de courants. Bien entendu, une prise de courant européenne et américaine n'ont pas les mêmes caractéristiques électriques, c'est pourquoi chaque prise, même si elle *hérite* d'une même classe générique, pourra disposer de ses propres attributs ou méthodes.

Le polymorphisme de fonctions

Le concept de polymorphisme peut également s'appliquer aux fonctions. Ainsi, l'exemple de la prise de courant est toujours pertinent : supposons que nous devions l'implémenter. Nous aurions une fonction powerOn(), qui lorsqu'elle est appelée, fournit du courant à l'appareil connecté. L'implémentation de cette méthode devra être différente entre une prise de type européenne, et une prise de type américaine, car le courant, la tension et la fréquence sont différents d'une prise à l'autre.

À nouveau, l'exemple des animaux faisait usage du polymorphisme de fonction : l'appel à la fonction speak() était différent selon l'objet depuis lequel l'appel à la fonction était réalisé.

Exemple

```
1  class PowerPlug:
2      voltage = 0
3      frequency = 0
4
5      def powerOn(self):
6          return (self.voltage, self.frequency)
7
8  class USPowerPlug(PowerPlug):
9      voltage = 110
10     frequency = 60
```

Tous droits réservés à STUDI - Reproduction interdite

```
11
12 class EUROPowerPlug(PowerPlug):
13     voltage = 220
14     frequency = 50
15
16 firstPowerPlug = USPowerPlug()
17 secondPowerPlug = EUROPowerPlug()
18
19 (resultingVoltage, resultingFrequency) = firstPowerPlug.powerOn()
20
21 print(resultingFrequency, resultingVoltage)
22
23 (resultingVoltage, resultingFrequency) = secondPowerPlug.powerOn()
24
25 print(resultingFrequency, resultingVoltage)
```

Remarque **Le Duck-Typing**

Il existe deux approches en termes de typage de variables.

- **Typage nominatif** : c'est l'approche la plus classique, deux objets sont du même types car ils sont définis comme tels (deux entiers, deux strings, etc.). En python, il s'agit d'un typage dynamique et implicite, c'est-à-dire que l'interpréteur 'comprendra' tout seul de quel type il s'agit en fonction des manipulations faites avec cet objet.
- **Duck-typing** : Une autre approche consiste à dire que deux objets sont « *équivalents* » non pas s'ils ont le même type, mais s'ils répondent aux mêmes méthodes. Le nom « *duck-typing* » provient du fait que « *Si cela marche comme un canard, et que cela fait le bruit du canard, alors ça ne peut être qu'un canard* ».

Exemple **Duck Typing**

```
1 class Canard:
2     def voler(self):
3         return "Je suis un canard, je peux voler !"
4
5 class Avion:
6     def voler(self):
7         return "Je suis un avion, je peux également voler !"
8
9 def faire_voler(objet_volatile):
10     print(objet_volatile.voler())
11
12
13 canard = Canard()
14 avion = Avion()
15
16 faire_voler(canard)      # Affiche : Je suis un canard, je peux voler !
17 faire_voler(avion)       # Affiche : Je suis un avion, je peux également voler !
18
19 # canard et avion sont par définition deux classes différentes, mais elles peuvent s'employer
   de la même façon
```

B. Exercice : Quiz

[solution n°2 p.16]

Question 1

Tous droits réservés à STUDI - Reproduction interdite

Qu'est-ce que le polymorphisme en programmation orientée objet ?

- ☐ La capacité d'une classe à changer d'attributs
- ☐ La capacité d'un objet à prendre plusieurs 'formes' d'implémentation
- ☐ La capacité d'une fonction à adapter son comportement en fonction de ses arguments
- ☐ La capacité d'une classe à hériter des méthodes d'une autre classe

Question 2

Comment est réalisé le polymorphisme avec des fonctions en Python?

- ○ En utilisant le mot-clé 'polymorph'
- ○ En définissant plusieurs fois la même fonction avec des arguments différents
- ○ En utilisant une fonction avec n'importe quel type d'objet qui prend en charge les opérations utilisées par la fonction
- ○ En utilisant le mot-clé 'function' avant la définition de la fonction

Question 3

Comment est réalisé le polymorphisme d'objets en Python ?

- ○ En utilisant le mot-clé 'polymorph'
- ○ En utilisant la notion d'héritage des classes
- ○ En utilisant des classes qui partagent une structure identique et ont les mêmes méthodes et attributs
- ○ En utilisant le mot-clé 'class' avant la définition de la classe

Question 4

Qu'est-ce que le « *Duck Typing* » en Python ?

- ○ Le fait que Python est un langage à typage dynamique
- ○ Le fait que Python ne se préoccupe que du comportement de l'objet avec lequel il travaille
- ○ Le fait que Python ne supporte pas le polymorphisme
- ○ Le fait que Python ne supporte pas l'héritage

Question 5

Quel est le lien entre polymorphisme et héritage ?

- ☐ Il n'y en n'a pas
- ☐ Le polymorphisme peut utiliser la notion d'héritage
- ☐ Ce sont deux termes qui désignent exactement la même chose
- ☐ Il est possible de faire du polymorphisme sans faire appel à l'héritage

Tous droits réservés à STUDI - Reproduction interdite

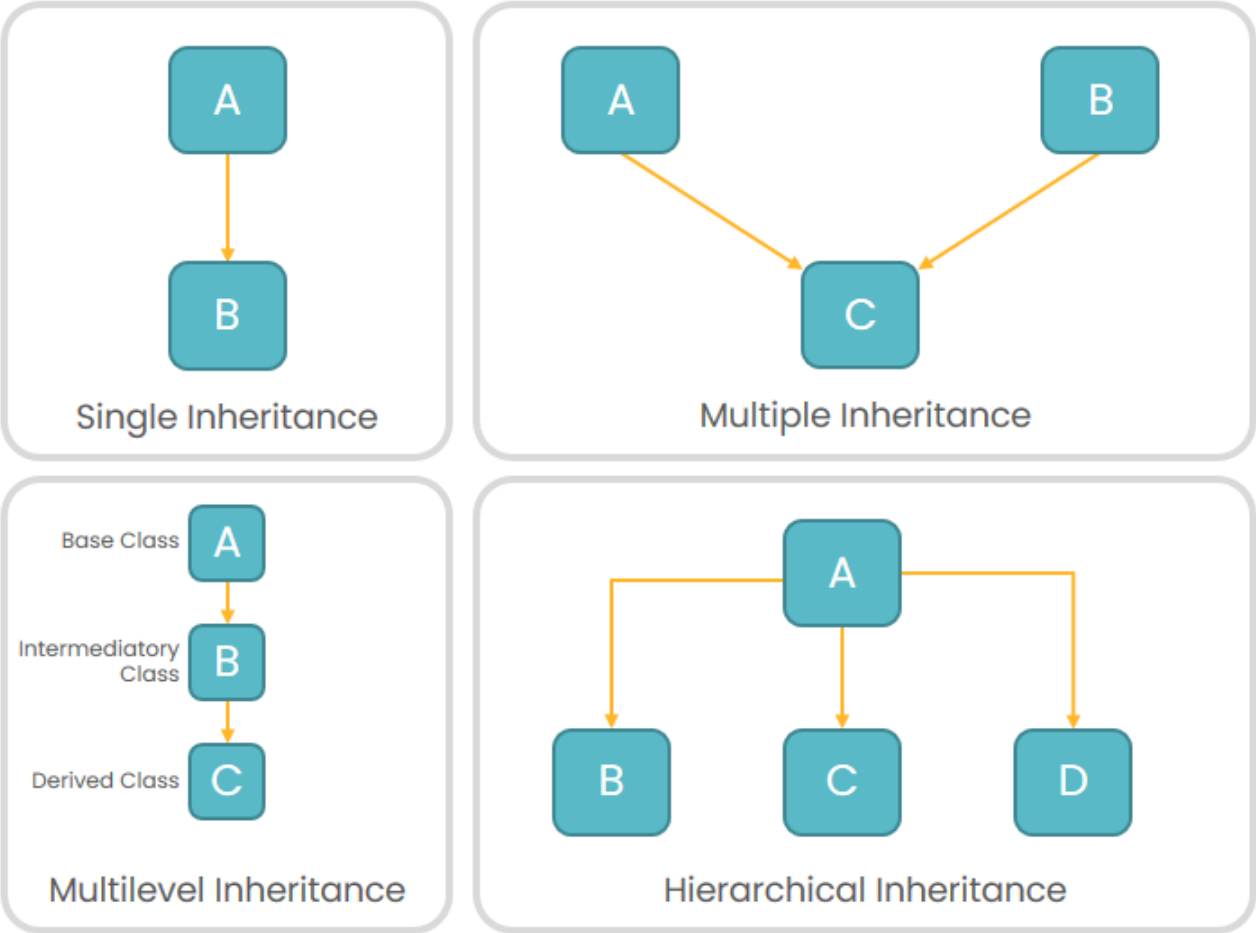
IV. Notions avancées d'héritage

A. Les types d'héritage

Les différents types d'héritage

Il existe plusieurs formes d'héritage :

- **Héritage simple** : c'est celui que nous avons mis en œuvre dans les exemples jusqu'à maintenant. Une classe enfant hérite d'une autre classe parent, tout simplement.
- **Héritage multiple** : une classe enfant hérite de plusieurs autres classes parentes. Ceci signifie que la classe enfant regroupe tous les attributs et méthodes de ses classes parentes.
- **Héritage multi-niveaux** : une classe enfant hérite d'une classe parent, qui elle-même hérite d'une classe parent, et ainsi de suite.
- **Héritage hiérarchique** : plusieurs classes enfants héritent d'une seule classe parent. C'est le cas de notre exemple sur les animaux abordé précédemment.



Bien que se présentant sous plusieurs formes, toutes ces variantes d'héritage font appel à un seul et même principe. La façon de le mettre en œuvre ou de le définir est identique, seule l'architecture du programme varie entre ces formes d'héritage.

Tous droits réservés à STUDI - Reproduction interdite

Remarque **Héritage multiple**

Il est pertinent de se demander comment se comporte l'héritage multiple si plusieurs classes parentes contiennent les mêmes définitions d'attributs ou de méthodes. Dans ce cas de figure, l'héritage se fera par priorité, comme illustré dans l'exemple suivant :

Exemple **Héritage multiple et priorité**

```
1 class A:
2     myAttribute = "Class A attribute"
3
4     def printAttribute(self):
5         print(self.myAttribute)
6
7
8 class B:
9     myAttribute = "Class B attribute"
10
11    def printAttribute(self):
12        print(self.myAttribute)
13
14
15 class C(A, B):
16    pass
17
18
19 myInstance = C()
20 myInstance.printAttribute()
```

V. L'essentiel

Dans la maîtrise de Python, la compréhension profonde de l'héritage et du polymorphisme est une étape clé. L'héritage, qui réside au cœur de la programmation orientée objet, représente un mécanisme permettant à une classe d'acquérir les attributs et méthodes d'une autre. Que vous définissiez une classe parente et sa descendance, ou que vous utilisiez des attributs et méthodes hérités, chaque détail compte pour un codage efficace et structuré. Et parfois, vous devrez aller au-delà de l'héritage simple et surcharger les méthodes et attributs pour adapter leur comportement à des besoins spécifiques.

À cela s'ajoute le concept de polymorphisme, un autre pilier de la programmation orientée objet. Ce principe permet à un objet de prendre différentes formes, facilitant ainsi une utilisation générale des fonctions et méthodes avec différents types de données. Que ce soit avec des fonctions, des classes, ou dans le contexte de l'héritage, le polymorphisme offre une flexibilité précieuse dans votre code.

Héritage et polymorphisme sont deux notions fortement liées, et qui sont très souvent employées dans tout type de développement orienté objet.

VI. Auto-évaluation

A. Exercice

Vous travaillez sur un projet Python où vous devez concevoir un système simple de gestion de la bibliothèque. Vous avez deux types d'objets à gérer : Livre et Magazine. Les deux ont des attributs communs, mais ils ont également des spécificités. En outre, vous voulez pouvoir traiter tous ces objets de manière uniforme, indépendamment de leur type spécifique.

Tous droits réservés à STUDI - Reproduction interdite

Question 1

[solution n°3 p.17]

Écrivez le code pour une classe mère « *Item* » et deux classes filles « *Livre* » et « *Magazine* ». Chaque item a un titre et un auteur, mais les livres ont également un attribut 'nombre_pages' et les magazines un attribut 'périodicité'.

Question 2

[solution n°4 p.18]

Écrivez une fonction qui accepte un objet Item et imprime ses informations. Cette fonction doit être capable de gérer à la fois les objets Livre et Magazine.

Question 3

[solution n°5 p.18]

Créez un objet Livre et un objet Magazine et utilisez la fonction 'afficher_info()' pour afficher leurs informations.

B. Test

Exercice 1 : Quiz

[solution n°6 p.18]

Question 1

Qu'est-ce qui est vrai concernant l'héritage en Python ?

- ○ Il n'y a pas d'héritage en Python
- ○ En Python, une classe peut hériter de plusieurs classes
- ○ En Python, une classe ne peut hériter que d'une seule classe
- ○ Python ne permet pas l'héritage de méthodes

Question 2

Quelle méthode serait surchargée pour définir le comportement d'une instance de classe lorsqu'elle est utilisée avec la fonction intégrée print() ?

- ○ __print__()
- ○ __str__()
- ○ __display__()
- ○ __show__()

Question 3

Qu'est-ce que le polymorphisme en Python ?

- ○ C'est une technique qui permet à un objet de prendre plusieurs formes
- ○ C'est une technique qui permet à un objet de changer de classe
- ○ C'est une technique qui permet à une classe de changer de forme
- ○ C'est une technique qui force un objet à rester dans une seule forme

Question 4

En Python, comment définiriez-vous une classe Enfant qui hérite d'une classe Parent ?

- ○ class Enfant(Parent): pass
- ○ class Enfant extends Parent: pass
- ○ class Enfant extends Parent: pass
- ○ class Parent -> Enfant: pass

Tous droits réservés à STUDI - Reproduction interdite

Question 5

Lors d'un héritage multi-niveaux, la dernière classe enfant peut accéder…

- Aux attributs et aux méthodes de son premier parent
- Aux attributs et aux méthodes de toutes les classes dont elle descend
- Uniquement aux méthodes de toutes les classes dont elle descend
- A ses propres attributs et méthodes, mais pas ceux de ses parents, car il s'agit d'une forme spécifique d'héritage

Solutions des exercices

Tous droits réservés à STUDI - Reproduction interdite

Exercice p. 5 Solution n°1

Question 1

Qu'est-ce que l'héritage en programmation orientée objet ?

- ○ Un mécanisme permettant de créer des classes
- ◉ Un mécanisme permettant à une classe d'acquérir les propriétés et méthodes d'une autre classe
- ○ Un mécanisme permettant d'ajouter des attributs à une classe
- ○ Un mécanisme permettant d'ajouter des méthodes à une classe

L'héritage est un principe fondamental de la programmation orientée objet qui permet à une classe d'acquérir les propriétés et méthodes d'une autre classe.

Question 2

Quels sont les avantages principaux de l'héritage en Python ?

- ☐ Une meilleure performance d'exécution
- ☐ La modularité
- ☑ La réutilisation du code et sa structuration
- ☑ La simplification du code

L'héritage en Python permet de réutiliser le code existant et d'organiser le code de manière plus logique et structurée. On peut également dire que l'héritage permet de simplifier la structure du code.

Question 3

Comment une classe hérite-t-elle d'une autre classe en Python ?

- ○ En utilisant le mot-clé 'extends'
- ◉ En indiquant la classe parent entre parenthèses dans la définition de la nouvelle classe
- ○ En utilisant le mot-clé 'inherit'
- ○ En indiquant la classe parent après un double point dans la définition de la nouvelle classe

En Python, une classe peut hériter d'une autre en mentionnant le nom de la classe parent entre parenthèses lors de la définition de la nouvelle classe.

Question 4

Qu'est-ce que la surcharge de méthode en Python ?

- ◉ Modifier le comportement d'une méthode héritée dans la classe enfant
- ○ Ajouter une nouvelle méthode à une classe existante
- ○ Supprimer une méthode d'une classe existante
- ○ Rendre une méthode héritée privée dans la classe enfant

La surcharge de méthode en Python est le processus par lequel on change le comportement d'une méthode héritée dans une classe enfant

Tous droits réservés à STUDI - Reproduction interdite

Question 5

Comment fait-on référence aux méthodes ou aux attributs du parent dans la classe enfant ?

- ○ Ce n'est pas possible
- ○ En indiquant le nom de la classe parent suivi d'un point : 'class.__'
- ○ Cela n'est pas nécessaire, on peut simplement écrire le nom de la méthode ou de l'attribut
- ◉ En utilisant le mot clé 'super()'

Le mot-clé 'super()' permet de faire référence à l'objet parent. De cette manière, il est possible d'accéder à ses attributs ainsi qu'à ses méthodes.

Exercice p. 8 Solution n°2

Question 1

Qu'est-ce que le polymorphisme en programmation orientée objet ?

- ☐ La capacité d'une classe à changer d'attributs
- ☑ La capacité d'un objet à prendre plusieurs 'formes' d'implémentation
- ☑ La capacité d'une fonction à adapter son comportement en fonction de ses arguments
- ☐ La capacité d'une classe à hériter des méthodes d'une autre classe

Le polymorphisme se réfère à la capacité d'un objet à prendre différentes formes. Cela signifie qu'une méthode ou une fonction peut être utilisée de manière indifférenciée avec divers types de données.

Question 2

Comment est réalisé le polymorphisme avec des fonctions en Python?

- ○ En utilisant le mot-clé 'polymorph'
- ○ En définissant plusieurs fois la même fonction avec des arguments différents
- ◉ En utilisant une fonction avec n'importe quel type d'objet qui prend en charge les opérations utilisées par la fonction
- ○ En utilisant le mot-clé 'function' avant la définition de la fonction

Le polymorphisme peut être réalisé en Python grâce aux fonctions. Une fonction peut être employée avec tout type d'objet qui autorise les opérations effectuées par cette fonction. Attention cependant, définir plusieurs fois la même fonction mais avec des arguments différents relève plutôt de la surcharge et non du polymorphisme (même si les concepts sont liés).

Question 3

Comment est réalisé le polymorphisme d'objets en Python ?

- ○ En utilisant le mot-clé 'polymorph'
- ○ En utilisant la notion d'héritage des classes
- ◉ En utilisant des classes qui partagent une structure identique et ont les mêmes méthodes et attributs
- ○ En utilisant le mot-clé 'class' avant la définition de la classe

Tous droits réservés à STUDI - Reproduction interdite

Des objets qui implémentent les mêmes attributs et les mêmes méthodes sont polymorphes, c'est-à-dire qu'ils ont la même forme, mais qu'ils ne sont pas forcément identiques. Attention, ces objets peuvent évidemment implémenter les méthodes différemment, tant que les noms des méthodes sont identiques, ils ont la même « *forme* ».

Question 4

Qu'est-ce que le « *Duck Typing* » en Python ?

- ○ Le fait que Python est un langage à typage dynamique
- ◉ Le fait que Python ne se préoccupe que du comportement de l'objet avec lequel il travaille
- ○ Le fait que Python ne supporte pas le polymorphisme
- ○ Le fait que Python ne supporte pas l'héritage

Le « *Duck Typing* » est une philosophie de typage propre à Python qui ne se préoccupe pas du type d'objet avec lequel nous travaillons, mais plutôt de son comportement. En d'autres termes, il est plus important de savoir ce qu'un objet peut faire (quelles méthodes il possède ou quelles opérations il supporte), plutôt que de connaître sa classe spécifique ou son type de données. Cette approche apporte une grande flexibilité à Python.

Question 5

Quel est le lien entre polymorphisme et héritage ?

- ☐ Il n'y en n'a pas
- ☑ Le polymorphisme peut utiliser la notion d'héritage
- ☐ Ce sont deux termes qui désignent exactement la même chose
- ☑ Il est possible de faire du polymorphisme sans faire appel à l'héritage

Le polymorphisme peut être réalisé en créant plusieurs enfants d'une même classe parent. C'est l'exemple du chat, du chien et de la vache. Il s'agit de différentes formes d'animaux, répondant à des méthodes similaires. Par ailleurs, il est également possible de définir deux classes avec des méthodes identiques, sans qu'elles héritent d'une classe parente.

p. 12 **Solution n°3**

```
1 class Item:
2     def __init__(self, titre, auteur):
3         self.titre = titre
4         self.auteur = auteur
5
6
7 class Livre(Item):
8     def __init__(self, titre, auteur, nombre_pages):
9         super().__init__(titre, auteur)
10        self.nombre_pages = nombre_pages
11
12
13 class Magazine(Item):
14    def __init__(self, titre, auteur, periodicite):
15        super().__init__(titre, auteur)
16        self.periodicite = periodicite
```

Tous droits réservés à STUDI - Reproduction interdite

p. 12 Solution n°4

Pour ce faire, il suffit de définir une fonction « *afficher_info(item)* ». Cette fonction peut prendre n'importe quel item (Livre ou Magazine) en entrée, et faire appel à une fonction « *get_infos_specifiques()* » qui est différente pour chaque type d'objet (surcharge de méthode) :

```
1 class Item:
2     def __init__(self, titre, auteur):
3         self.titre = titre
4         self.auteur = auteur
5 
6     def get_infos_specifiques(self):
7         return (self.titre, self.auteur)
8 
9 class Livre(Item):
10     def __init__(self, titre, auteur, nombre_pages):
11         super().__init__(titre, auteur)
12         self.nombre_pages = nombre_pages
13 
14     def get_infos_specifiques(self):
15 
16         return (f" {super().get_infos_specifiques()} Nombre de pages : {self.nombre_pages}")
17 
18 class Magazine(Item):
19     def __init__(self, titre, auteur, periodicite):
20         super().__init__(titre, auteur)
21         self.periodicite = periodicite
22 
23     def get_infos_specifiques(self):
24         return (f" {super().get_infos_specifiques()} {self.periodicite}")
25 
26 def afficher_info(item):
27     print(f"{item.get_infos_specifiques()}")
```

p. 12 Solution n°5

Nous créons simplement deux objets : un livre et un magazine. Ensuite, nous appelons la fonction « *afficher_info()* » pour chaque objet :

```
1 livre = Livre("Le Petit Prince", "Antoine de Saint-Exupéry", 96)
2 magazine = Magazine("Nature", "John Smith", "Mensuel")
3 
4 afficher_info(livre)
5 afficher_info(magazine)
```

Exercice p. 12 Solution n°6

Tous droits réservés à STUDI - Reproduction interdite

studi

Question 1

Qu'est-ce qui est vrai concernant l'héritage en Python ?

- ○ Il n'y a pas d'héritage en Python
- ◉ En Python, une classe peut hériter de plusieurs classes
- ○ En Python, une classe ne peut hériter que d'une seule classe
- ○ Python ne permet pas l'héritage de méthodes

En Python, une classe peut hériter de plusieurs classes, c'est ce qu'on appelle l'héritage multiple. De plus, l'héritage en Python permet non seulement d'hériter des attributs, mais également des méthodes de la classe parente.

Question 2

Quelle méthode serait surchargée pour définir le comportement d'une instance de classe lorsqu'elle est utilisée avec la fonction intégrée print() ?

- ○ __print__()
- ◉ __str__()
- ○ __display__()
- ○ __show__()

En Python, la méthode spéciale __str__() est utilisée pour définir le comportement d'une instance de classe lorsqu'elle est convertie en chaîne de caractères, par exemple lorsque vous utilisez la fonction print()

Question 3

Qu'est-ce que le polymorphisme en Python ?

- ◉ C'est une technique qui permet à un objet de prendre plusieurs formes
- ○ C'est une technique qui permet à un objet de changer de classe
- ○ C'est une technique qui permet à une classe de changer de forme
- ○ C'est une technique qui force un objet à rester dans une seule forme

Le polymorphisme en Python fait référence à la capacité d'un objet à prendre plusieurs formes. C'est un concept clé de la programmation orientée objet qui facilite la réutilisation du code et le rend plus flexible.

Question 4

En Python, comment définiriez-vous une classe Enfant qui hérite d'une classe Parent ?

- ◉ class Enfant(Parent): pass
- ○ class Enfant extends Parent: pass
- ○ class Enfant extends Parent: pass
- ○ class Parent -> Enfant: pass

En Python, pour définir une classe Enfant qui hérite d'une classe Parent, nous utilisons la syntaxe « *class Enfant(Parent): pass* ». Ici, « *pass* » est utilisé comme un espace réservé, car Python attend un bloc de code après les deux points.

Tous droits réservés à STUDI - Reproduction interdite

Question 5

Lors d'un héritage multi-niveaux, la dernière classe enfant peut accéder…

- ○ Aux attributs et aux méthodes de son premier parent
- ◉ Aux attributs et aux méthodes de toutes les classes dont elle descend
- ○ Uniquement aux méthodes de toutes les classes dont elle descend
- ○ A ses propres attributs et méthodes, mais pas ceux de ses parents, car il s'agit d'une forme spécifique d'héritage

Un héritage multi-niveaux n'est rien de plus qu'un héritage par « *succession* ». La classe enfant tout en bas de l'héritage peut donc accéder à tous les attributs et méthodes de l'ensemble de ses classes parents.

Tous droits réservés à STUDI - Reproduction interdite